0.1 Introduction

The Critical Path Toolkit consumes the unified graph object of Chapter 3 directly and involves no decomposition. Every analysis in it is a pair of linear-time passes over the layered iteration sets, so the Network Decomposition Module of Chapter 4 is bypassed entirely.

The quantities this toolkit propagates are the ones that accumulate along dependency paths. A duration, a cost, a risk factor and a load all compound as they travel, and a network carrying such a quantity poses the same pair of questions whatever the quantity is. One is extremal, asking for the best or worst value a complete chain of dependencies can produce. The other concerns room, asking how far an individual component can move before that extremal answer changes. The first is settled by a forward pass. The second is where the analysis actually lives, because room is what criticality, bottlenecks and priorities are made of, and it exists only where the arithmetic of the quantity can be run backwards. This chapter develops that backward reading and its price under uncertainty.

0.2 Scheduling Analysis on Networks

The critical path method models a project as a network of activities and computes the longest path through it. The project cannot finish earlier than that path allows, the activities on it have no room to slip, and every other activity carries a float equal to how far it can slip before it starts to matter. The method and the float calculus date to the late 1950s [1], and the companion question of duration uncertainty was raised almost immediately: PERT replaced each fixed duration with a three-point estimate and a distributional assumption [2], and its descendants in modern practice replace the assumption with Monte Carlo sampling. Distribution-free treatment of the same uncertainty came much later. When each duration is known only as an interval, the forward quantities remain easy, but the backward ones change character entirely. Deciding whether an activity is possibly critical is NP-complete and float bounds are NP-hard [3], even on planar networks [4]. Exact methods exist for latest starting times and one of the two float bounds [5], the consolidated picture is given by Fortin, Zieliński, Dubois and Fargier [6], and the recognised tractable class is the series–parallel networks.

None of the machinery is specific to time. The forward recursion combines incoming values and compounds them along edges, and any quantity with those two operations propagates the same way. With combination as maximum and compounding as addition the recursion is the critical path method. With other pairs it is cost roll-up, risk scaling or route selection, and the algebra of such systems is well established: max-plus linear systems theory treats the classical forward and backward passes as linear algebra over

an idempotent semiring, with the backward pass arising as residuation [7]. The lesson of that algebra is a discipline. A forward recursion exists for any pair of operations, but a backward one is a property the pair either has or lacks, and an analysis that reports slack without checking has manufactured a number. Uncertainty sharpens the same discipline from another side, because the interval results above draw their line between easy and hard exactly where monotonicity is lost, and the structure that loses it turns out to be the reconvergence that Chapter 4 measures for probability.

0.3 Analysis Modes

Whether the room question has an answer is a property of the operator pair, and it has one in exactly two situations. When combination is an order, maximum or minimum, a bound on the combined value binds every argument, and when compounding is monotone with a residual the bound inverts across each edge, so a constraint of the form finish by K travels backwards through the network the same way values travel forwards. That is residuation in an idempotent semiring [7], and it is what the classical backward pass has always been. Summation is different in kind. A total is not a deadline and no residuation exists, but a summed system is linear, and a linear map has an adjoint. The backward object of an accumulation analysis is therefore a sensitivity, and the room it measures is an allowance against a stated budget rather than a slack against a deadline. A pair with neither structure supports a forward recursion and nothing else, and any margin reported for it would be invention. A configuration of the toolkit, called a mode, is an operator pair together with whatever backward semantics its algebra supports, and Table 1 lists the four the framework provides.

Table 1: The shipped modes. Every row carries a defined backward semantics; the margin column names what the backward pass reports.

Mode	$\oplus$ / $\otimes$	Backward mechanism	Margin
LongestPath	max / +	residuation (min / $-$)	slack
ShortestPath	min / +	dual residuation (max / $-$)	margin over optimum
MaxScaling	max / $\times$	residuation (min / $\div$)	ratio slack
Accumulation	sum / +	adjoint (path multiplicities)	allowance

The four margins in the table are two objects, not four. The three order-based modes all report the same thing, the distance between the best complete path and the best path forced through the node in question, expressed as a difference for the additive pairs and as a ratio for the multiplicative one, and in each case the zero set is the critical structure of the analysis. The allowance is the linear system's counterpart. Because a node's value reaches the target once per route, its influence on the total is a path count

rather than a path choice, and the room it has under a budget is the remaining headroom shared across those routes. Running a scheduling backward pass over a summed quantity instead produces figures that answer no question at all, which is why no slack appears in the accumulation row. The water network of Section 0.7.1 shows what the allowance says in its place.

0.4 The Two Kernels

The backward pass is not a second algorithm. For the order-based modes one fold serves both directions. Read forwards over the layered iteration sets it computes, for every node v with parent set $\mathrm{pa}(v)$, duration d_v and edge values w_{uv},

$$F_v \;=\; \Big(\oplus_{u\in \mathrm{pa}(v)}\, F_u \otimes w_{uv}\Big) \otimes d_v,$$

the best value a chain of dependencies can deliver into each node, with sources seeded from the initial value. Read over the reversed graph the same fold computes

$$R_v \;=\; \oplus_{s\in \mathrm{succ}(v)}\, \big(R_s \otimes d_s\big) \otimes w_{vs},$$

the best completion from each node onwards, with sinks seeded at the neutral element. The two arrays meet in the middle. Their compound $F_v \otimes R_v$ is the best complete path forced through v, the project value P is the best complete path free of any such constraint, and the margin of v is the gap between the two. Slack, in this reading, is the cost of insisting on a node. The textbook schedule quantities are recoveries from the same two arrays, early start as $F_v - d_v$ and late finish as $P - R_v$, with late start and total float following by the same subtractions. On the four-node network of Figure 1, with durations $1, 5, 2, 1$ and no edge delays, the longest reading completes in 7 along $1{\to}2{\to}4$ while node 3 carries slack 3, and the shortest reading completes in 4 along $1{\to}3{\to}4$ while node 2 carries margin 3. The same inputs answer opposite questions with opposite critical chains.

Accumulation replaces the order by a sum, and the meeting in the middle changes meaning. The total at a target t expands to $\sum_v m_v d_v + \sum_{(u,v)} m_v w_{uv}$, where m_v counts the directed routes from v to the target, so a node influences the total not through one best path but once per route, and the reverse traversal that computes the counts, seeded with $m_t = 1$, is the adjoint of the forward map. The count is the derivative of the total with respect to the node's value, the product $m_v d_v$ is the node's share of the total, and under a budget B the room left to the node is $(B - F_t)/m_v$, headroom divided across its routes. On the running example the total at node 4 is 10 and node 1 counts twice, once through each branch. Both kernels touch each edge a constant number of times, so every

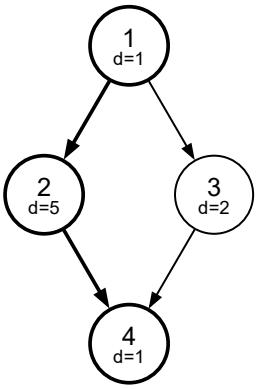

Figure 1: The running example. Node durations $1, 5, 2, 1$; LongestPath is critical through nodes $1, 2, 4$ and ShortestPath through $1, 3, 4$.

mode costs $O(|V| + |E|)$ whatever the operators are.

0.5 Interval Analysis

Uncertainty divides the toolkit's quantities along a single line, and the line is monotonicity. Every forward quantity grows when any input grows, so with interval inputs its exact bounds are two crisp runs of the scalar kernel, one at the lower corner of the input box and one at the upper, and interval arithmetic never enters. Margins sit on the other side of the line. A margin is a difference of two monotone quantities that share their inputs, it is not monotone in anything, and the hardness results of Section 0.2 live precisely there. Running the scalar algorithms on an overloaded interval type would blur this line rather than respect it, ranking overlapping intervals through a partial order that cannot rank them and subtracting dependent quantities as though they were independent. The toolkit instead computes each interval quantity by the scheme its structure permits. Forward quantities are exact for the price of two runs. Margins come either as a sound enclosure derived from those exact bounds, declared conservative in the result it returns, or exactly, and an exact value and an enclosure are never allowed to look alike.

0.6 Exact Interval Floats: the Domination Split

Throughout this section the mode is LongestPath, durations d_u lie in intervals, delays are fixed, and $f_v = P - \text{through}_v \geq 0$ is the float of node v. An interval-valued delay reduces to this setting by subdividing its edge with a node that carries the delay as its duration. The question is the exact range of f_v over the box of duration configurations.

Proposition 1 (Corner sufficiency)**.** *With all other durations fixed, f_v is a monotone function of any single duration d_u. Consequently the extremes of f_v over the box are attained at corner configurations.*

Proof. A path visits a node at most once, so as a function of $x = d_u$ every path length is either constant or affine with slope one. Hence $P(x) = \max(C_0, C_1 + x)$, where C_1 is the best path through u and C_0 the best avoiding it, and likewise $\text{through}_v(x) = \max(A_0, A_1 + x)$ over the paths through v. Each has one breakpoint with slope 0 then 1. If P's breakpoint comes first their difference has slope pattern $0, +1, 0$, otherwise $0, -1, 0$, and in both cases f_v is monotone in x on the whole line. For the corner claim, push any coordinate to whichever endpoint does not worsen the objective, which monotonicity permits, and repeat per coordinate until the configuration is a corner. □

Lemma 2 (Incomparable coordinates)**.** *If no complete path contains both u and v, then f_v is nondecreasing in d_u, whatever the other durations are.*

Proof. No through-v path contains u, so through_v does not depend on d_u, while P is nondecreasing in it. □

Lemma 3 (Dominated coordinates)**.** *If every complete path through u also passes v, and in particular if $u = v$, then f_v is nonincreasing in d_u, whatever the other durations are.*

Proof. Suppose increasing d_u increases P. The new maximiser must contain u, hence v, hence it is a through-v path, so through_v rises to equal P and the float drops to zero. If P does not increase, the float cannot rise because through_v is nondecreasing in every duration. □

The two lemmas cover every duration except those in the *bypass set*

$$H_v = \{\, u : \text{some complete path contains both } u \text{ and } v, \text{ and some complete path contains } u \text{ but not } v$$

the nodes that share a route with v yet can also route around it. Figure 2 shows the three classes around one node.

Theorem 4 (Exactness of the domination split)**.** *The maximum of f_v over the box is attained at a corner with every incomparable duration at its upper bound, every dominated duration at its lower bound, and the bypass durations at some corner of H_v. The minimum is attained dually. Enumerating the $2^{|H_v|}$ bypass corners with the remaining durations pinned therefore yields the exact float range of v.*

Proof. By Proposition 1 an optimum lies at a corner. The three classes partition the durations. At an optimal corner, moving an incomparable duration to its upper bound cannot decrease the maximum by Lemma 2, and moving a dominated duration to its lower bound cannot decrease it by Lemma 3, so some optimal corner has the pinned pattern and lies within the enumerated set. □

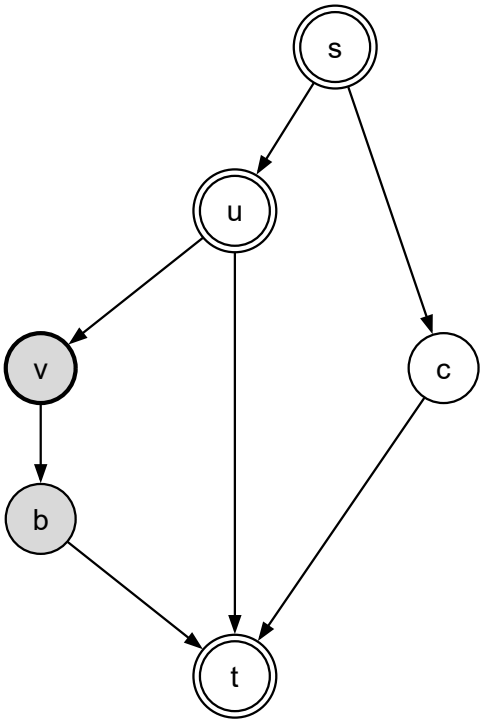

Figure 2: Duration classes relative to node v. Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v: they share routes with v and can also route around it.

The cost is $\sum_v 2^{|H_v|}$ crisp propagations against 2^k for exhaustive enumeration of all k interval durations, and the exponent has moved from the size of the problem to the reconvergent width around each node. One exhaustive sweep serves every node simultaneously, so the implementation runs whichever of the two is cheaper and is never worse than exhaustive enumeration. Some instances must remain hard, because the general problem is NP-hard [3], and the split shows exactly where the hardness sits: in networks whose bypass sets are large.

The bypass set is a reconvergence measure, and this is where the toolkit touches the structural theme of Chapter 4 without ever calling its module. A bypass node is precisely an N-shaped pattern in the reachability order, and the orders of two-terminal series–parallel networks are exactly the N-free orders [8].

Proposition 5 (Series–parallel case)**.** *In a two-terminal series–parallel network every pair of nodes on a common path is in the dominated relation and every other pair is incomparable, so H_v is empty for every v, and the split computes exact interval floats in two crisp runs per node.*

This recovers the known polynomiality of interval criticality on series–parallel networks as a special case, and it makes the kinship with probability precise. The conditioning sets of Chapter 4 and the bypass sets of this chapter vanish on exactly the same class of structures and grow with the same phenomenon. Reconvergence is the price of exactness in both analyses. The two objects are not the same, they index reconvergence per join and per node respectively, and neither computation uses the other. Whether the diamond spans can accelerate the split by sharing corner evaluations between nodes with overlapping bypass sets is an open engineering question.

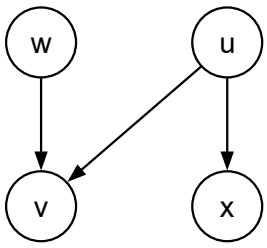

Figure 3: The N pattern that places u in a bypass set: u reaches v and also reaches a sink around it. Series–parallel orders contain no such pattern.

0.7 Case Studies

Every number in this section is produced by the toolkit and certified against an independent computation. The certification corpus contains ten networks. On the seven where full path enumeration is affordable, forward values, through-values, margins and critical sets of every mode agree with a brute-force path oracle to within 10^{-15}, and on the remaining three the results pass invariant and sampling checks. Interval results are certified against exhaustive corner enumeration driven by the same path oracle, and the domination split agrees with exhaustive enumeration everywhere both can run.

0.7.1 One network, four questions

The wastewater treatment network of 32 nodes and 66 edges shows how much one set of inputs can say. Its stage processing times and pipeline delays fix the minimum release timeline at 45 units, carried by the chain $3 \to 11 \to 19 \to 27$ with a second chain only 1.5 units behind (Table 2). Read for the opposite extreme, the same inputs put the fastest achievable delivery at 15 units, and not along one route. The twelve margin-zero stages and the eighteen tight dependencies between them form a system of tied routes drawing on all three sources, sharing no stage with the critical chain, and the nearest alternative stage sits 1.5 units behind the tie (Figure 4). The two extremal readings of one network differ not only in which stages they select but in the shape of what they select, a single thread against a redundant lattice.

Table 2: LongestPath on the water network: completion 45 along the critical chain, with the near-critical pair at slack 1.5.

Node	Completion	Slack
27	45.0	0
19	33.0	0
11	21.0	0
3	9.0	0
30	43.5	1.5
14	19.5	1.5

Read as load rather than schedule, the same network delivers 480 units to its busiest outlet, and the stages that drive that figure are not the ones the completion times point at. The largest completions sit downstream where load gathers, but the leverage sits upstream at stages 3, 6 and 5, each reaching the outlet along seven distinct routes, so a unit added at any of them arrives seven times. Under a budget ten percent above the current total those three stages can each grow by less than seven units, while stage 14, on two routes, has room for 24 (Table 3). Contribution and room order the stages

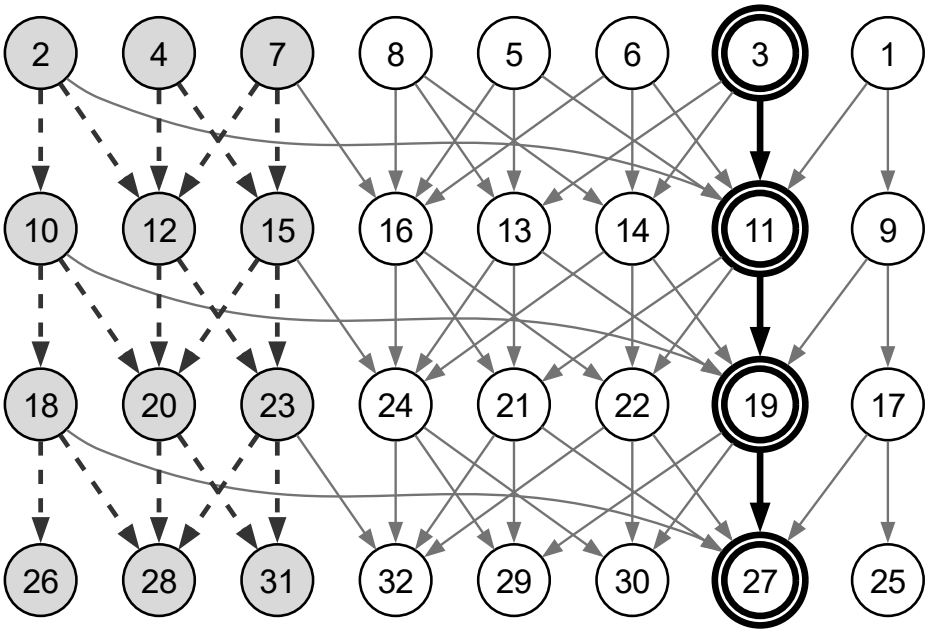

Figure 4: The water network read for both extremes. The bold double-bordered chain sets the minimum release timeline of 45 units. The shaded stages and dashed edges form the tied-fastest route system, every path through which completes in 15 units. The two readings share no stage.

differently, which is the information a capacity decision needs and which no scheduling slack could express.

Table 3: Accumulation on the water network under a budget of 528 units at outlet 27 (current total 480). Allowance is headroom divided by path multiplicity.

Node	Contribution	Multiplicity	Allowance
3	63	7	+6.9
6	56	7	+6.9
5	35	7	+6.9
1	30	5	+9.6
11	27	3	+16
14	16	2	+24

Uncertainty then tests how much of the deterministic story survives. With every one of the 32 stage durations widened to ±20 percent, exhaustive corner enumeration would require 2^{32} propagations, roughly 4.3×10^9. The domination split required 2,269,328, a factor of 1,892 fewer, with the largest bypass set holding 18 of the 32 durations, and completed in 135 seconds on commodity hardware. The answer changes how the deterministic result must be read. Under full uncertainty no stage is necessarily critical, and exactly nine of the 32 are possibly critical. The single deterministic critical chain dissolves into a nine-candidate set (Figure 5), and 23 stages are provably incapable of becoming critical anywhere in the uncertainty box. A Monte Carlo attack on the same bounds, fifty thousand sampled configurations, produced no violation and reached every bound at every node.

0.7.2 The most reliable route

The drone medical delivery network of 782 nodes, 2,789 edges, 33 launch sites and 323 destinations poses a multiplicative question, which route delivers with the highest end-to-end success. The factors are the study's own component success rates, node priors between 0.75 and 0.95 and per-link probabilities between 0.70 and 0.95, so nothing in the scenario is invented. The best route in the network compounds to a success factor of 0.787, and it is a single direct leg from launch site 380 to destination 393. That brevity is structural rather than accidental. The strongest leg the data permits compounds its link and node factors to 0.9025, so every leg a route adds costs at least 9.8 percent of its success, and no amount of component quality along a longer route can repay that arithmetic. Route brevity is the currency of the multiplicative reading, which nothing in an additive analysis of the same network would reveal.

Read per destination rather than globally, the forward values become a service-level map, and the map is unequal (Figure 6). The five best-served destinations sit within seven

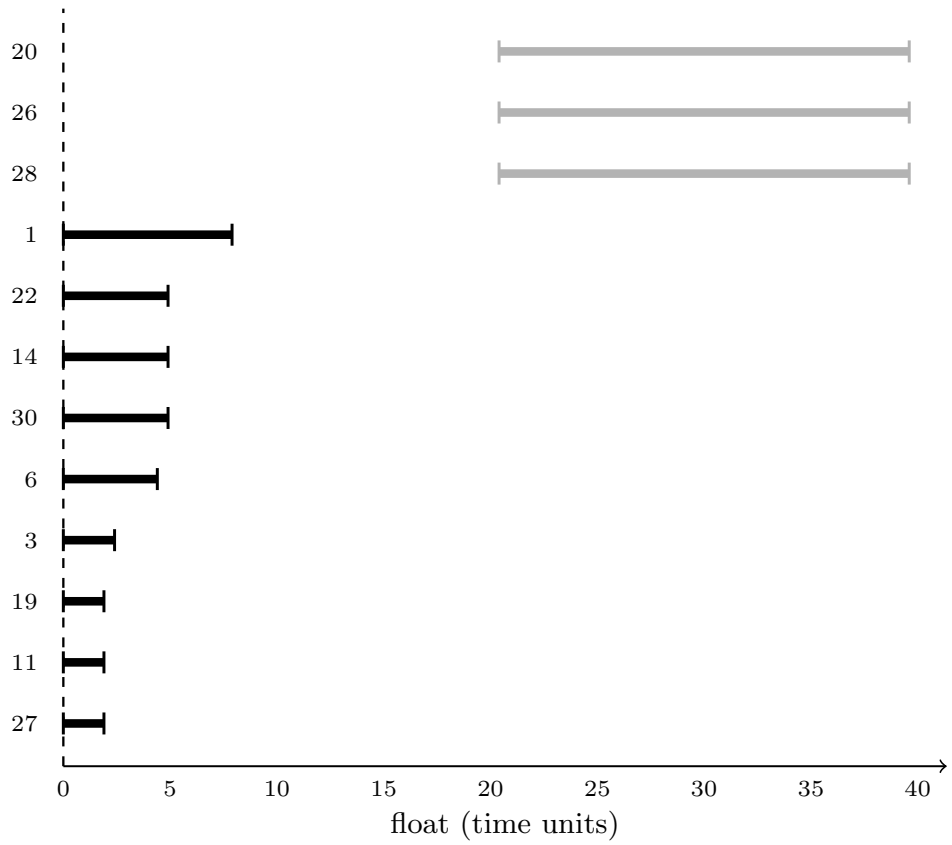

Figure 5: Exact float bounds on the water network with every duration at plus or minus 20 percent. Black bars are the nine possibly-critical stages, whose lower bounds reach zero. Grey bars are three of the 23 stages whose floats stay provably positive everywhere in the uncertainty box.

Table 4: The five best-served destinations by single-route success. At the other end of the scale, 104 of the 323 destinations fall below 0.5 on even their best route.

Destination	Best-route success
393	0.787
325	0.771
209	0.747
248	0.742
125	0.734

percent of the optimum (Table 4), while the weakest destination in the network can be reached with success 0.108 by its best route, and for 104 of the 323 destinations, roughly one in three, even the best single route succeeds less than half the time. No routing decision alone can serve a third of this network reliably, and where that matters the remedy is structural, stronger components on the thin corridors or genuinely redundant routes. Judging that redundancy is the neighbouring question, because a destination poorly served by its best single route may still be reliably reached when every route counts at once, which is the reachability reading of Chapter 5. The two readings separate route choice from system adequacy, and a delivery planner for this network needs both. Two thousand randomly sampled routes never exceeded the reported optimum and one reached it exactly, and the optimum itself is the product of three published input values, checkable by hand.

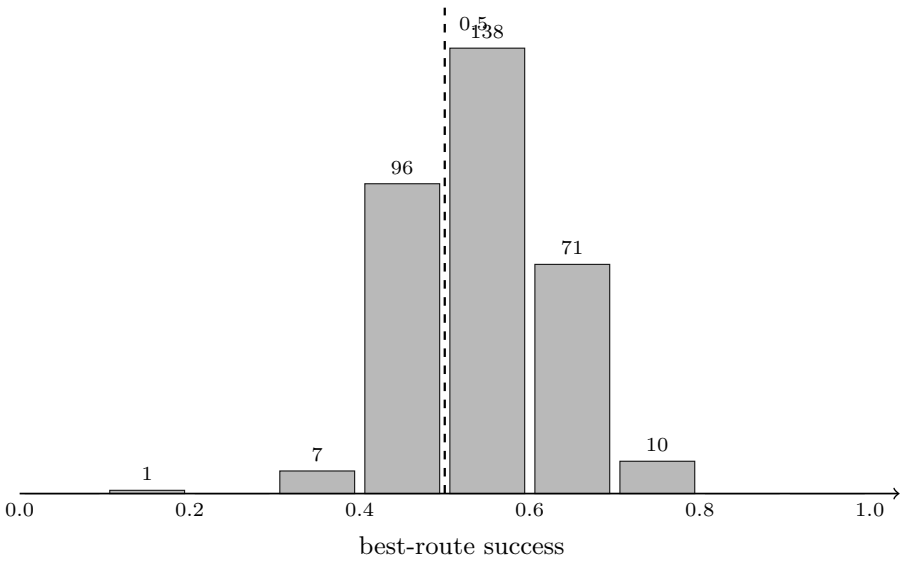

Figure 6: Best-route success across the 323 drone destinations. Left of the dashed line at 0.5 lie the 104 destinations that no single route serves reliably.

0.7.3 The boundary

Dense mesh reconvergence is where the split's advantage disappears. On a five-by-five grid DAG with all 25 durations interval-valued, the bypass sets hold up to 23 of the 25 durations and the split would need more propagations than one shared exhaustive sweep, which the implementation therefore uses instead. The hardness results guarantee that some regime behaves this way, and it is the same regime in which the probability toolkit's conditioning sets explode. Reconvergence density is the shared price axis of both chapters, and mapping the boundary is part of the result rather than a qualification of it. Timings are reported only as measured. The water analyses run in 53 microseconds per

pass for the additive modes and 7 for accumulation, the drone network in 1.8 milliseconds, and the exhaustive comparisons are quantified by run counts rather than by extrapolated wall-time.

0.8 Genericity

The boundary of the toolkit is drawn by the algebra rather than by the implementation. A new quantity joins by exhibiting its operator pair, and the pair earns its backward pass the way the shipped ones did, by carrying a residual or by being linear. The obligations are laws, verified on sampled values before a mode is accepted, so an extension can never silently acquire a backward pass its algebra does not support. The kernels are generic over the value they propagate, and interval support is a scheme wrapped around crisp runs rather than a property of a value type, which is what keeps its exactness claims provable.

The nearest extensions follow the existing arguments. The shortest-path dual of the domination split mirrors Section 0.6 and awaits only the line-by-line derivation. Interval-valued delays reduce to interval durations by subdividing their edges. Further out, neighbouring nodes have overlapping bypass sets and therefore overlapping corner evaluations, an economy of the kind the hash-consed store of Chapter 4 already practises for diamonds, and the exclusion predicate of that chapter's identification algorithm has in fact run with a zero-width interval definition during the development of the split, so the decomposition module's extension surface is known to support value semantics other than probability.

0.9 Summary

The toolkit reads one graph object in two directions. Forwards it computes the extremal value of any quantity that accumulates along dependency paths. Backwards it computes the room each component has before that value moves, wherever the quantity's algebra defines such a reading, which is the condition every shipped mode satisfies and every future mode must. The classical critical path method is the default instance rather than the design, and the sum family reports sensitivities and allowances because that is what its mathematics defines. Under interval uncertainty the forward quantities stay exact for the price of two crisp runs. The floats are exact through the domination split, whose correctness rests on three short proofs and whose cost grows with the reconvergent width around each node rather than with the number of uncertain inputs. That width stayed small enough on a real network to solve a 32-duration instance beyond the reach of exhaustive enumeration, it does not stay small on dense meshes, and it vanishes exactly on the series–parallel structures where the decomposition module of Chapter 4 would also

find nothing to decompose. Every number reported in this chapter is certified against an independent oracle, an exhaustive enumeration or a sampling attack.

Bibliography

[1] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318

[2] D. G. Malcolm, J. H. Roseboom, C. E. Clark, and W. Fazar, "Application of a technique for research and development program evaluation," *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: 10.1287/opre.7.5.646

[3] S. Chanas and P. Zieliński, "The computational complexity of the criticality problems in a network with interval activity times," *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: 10.1016/S0377-2217(01)00048-0

[4] S. Chanas and P. Zieliński, "On the hardness of evaluating criticality of activities in a planar network with duration intervals," *Operations Research Letters*, vol. 31, no. 1, pp. 53–59, 2003.

[5] D. Dubois, H. Fargier, and J. Fortin, "Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks," *Journal of Intelligent Manufacturing*, vol. 16, pp. 407–421, 2005. DOI: 10.1007/s10845-005-1654-5

[6] J. Fortin, P. Zieliński, D. Dubois, and H. Fargier, "Criticality analysis of activity networks under interval uncertainty," *Journal of Scheduling*, vol. 13, pp. 609–627, 2010. DOI: 10.1007/s10951-010-0163-3

[7] F. Baccelli, G. Cohen, G. J. Olsder, and J.-P. Quadrat, *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Chichester: John Wiley & Sons, 1992.

[8] J. Valdes, R. E. Tarjan, and E. L. Lawler, "The recognition of series parallel digraphs," *SIAM Journal on Computing*, vol. 11, no. 2, pp. 298–313, 1982. DOI: 10.1137/0211023